Project 2 Report: `schedsim`

CMPE 230 Systems Programming – Spring 2026

Yiğit Sarp Avcı, 2023400048
Doğukan Sungu, 2023400210

May 4, 2026

1 Project Overview

Our scheduler simulator (`schedsim`) is a self-contained x86-64 GNU Assembly program that reads a single input line from standard input, determines which CPU scheduling algorithm to use, parses the associated process descriptors, simulates each clock cycle of the scheduler, and writes the resulting execution timeline string to standard output.

The program supports five scheduling algorithms:

- **FCFS** – First Come First Serve (non-preemptive, arrival-based)
- **SJF** – Shortest Job First (non-preemptive, all arrive at time 0)
- **SRTF** – Shortest Remaining Time First (preemptive)
- **PF** – Priority First (preemptive, lower value = higher priority)
- **RR** – Round Robin (quantum-based, cyclic)

The overall execution flow is:

1. Read up to 255 bytes from `stdin` via the `sys_read` syscall.
2. Null-terminate the input buffer.
3. Parse the first token to determine the algorithm type.
4. Parse all process descriptors (and the quantum for RR) from the remaining input.
5. Dispatch to the appropriate scheduling routine.
6. Write the accumulated output buffer to `stdout`.
7. Exit via `sys_exit` with code 0.

2 Parsing Logic

2.1 Input Processing Strategy

The input line is a single string containing space-separated tokens. The first token identifies the algorithm (e.g., "FCFS", "SJF", "SRTF", "PF", "RR"). Each subsequent token is a process descriptor whose fields are separated by hyphens.

Our parsing operates in two phases:

1. **Algorithm detection**: We inspect the first one or two characters of the input buffer. The character 'F' indicates FCFS, 'P' indicates PF, 'R' indicates RR, and 'S' requires a second-character check ('J' for SJF, 'R' for SRTF). After identification, we advance the input pointer past the algorithm token and its trailing space.

2. **Process descriptor parsing**: Starting from the position after the algorithm token, we iterate over space-separated tokens. For each token, we extract the process ID (a single uppercase letter), skip the hyphen delimiter, and accumulate the burst time digit by digit. Depending on the algorithm type, we then extract additional fields (arrival time, priority) in the same manner.

For Round Robin, the last token in the input has no hyphen – it is the quantum value. We detect this by scanning ahead in the current token to check for the presence of a hyphen before deciding whether to parse it as a process descriptor or as the quantum.

2.2 Register-Level Parsing Design

During parsing:

- `rsi` serves as the **input pointer**, advancing character by character through the input buffer. It is loaded with the base address of `input_buf` and incremented after each character is consumed.

- `rax` is used as the **numeric accumulator**. For each digit character, we compute `rax = rax * 10 + (char - '0')` using the `imul` instruction for the multiply and an `add` for the digit value.

- `ecx` (the lower 32 bits of `rcx`) holds the current character being examined, loaded via `movzbl (%rsi), %ecx` which zero-extends the byte to avoid sign issues.

- `r12` serves as the **process counter**, tracking how many processes have been parsed so far.

- `r14` holds the **algorithm type code** (0–4) so that field parsing logic can branch accordingly.

- `rdi` is used as a **temporary base pointer** for storing parsed values into the appropriate arrays (e.g., `proc_burst`, `proc_arrival`) using scaled indexed addressing `(%rdi, %r12, 8)`.

Correctness is maintained by:

- Resetting `rax` to zero with `xor %rax, %rax` before each numeric field parse.
- Checking delimiter characters (hyphen, space, null, newline, carriage return) explicitly to terminate digit accumulation.
- Using separate parsing loops for each field, with clear transitions between them.

3 State Representation (Memory Layout)

3.1 Process Representation

Each process is represented using **parallel arrays** stored in the `.bss` section. Every array has space for 10 entries, each 8 bytes (quad-word) wide, for a total of 80 bytes per array:

- `proc_id[10]` – stores the ASCII character of the process ID (e.g., 65 for 'A'). Although only one byte is needed, we use 8-byte slots for uniform indexed addressing.
- `proc_burst[10]` – the original burst time of each process.
- `proc_arrival[10]` – the arrival time (set to 0 for SJF and RR where all processes arrive simultaneously).
- `proc_remaining[10]` – the remaining burst time, initialized to the burst time during parsing and decremented during simulation.
- `proc_priority[10]` – the priority value (used only by the PF algorithm).

This parallel-array design simplifies iteration: given a process index `i` in a register, we access any attribute via `base_address + i * 8` using x86-64 scaled indexing mode.

3.2 Global State

- `proc_count` (8 bytes) – the number of processes parsed from the input.
- `algo_type` (8 bytes) – an integer code identifying the algorithm (0=FCFS, 1=SJF, 2=SRTF, 3=PF, 4=RR).
- `quantum_val` (8 bytes) – the quantum value for Round Robin.
- `output_buf` (4096 bytes) – the timeline output buffer. Characters are appended sequentially.
- `output_len` (8 bytes) – the current number of characters in `output_buf`, used as the write index.

- `input_buf` (256 bytes) – the raw input read from stdin, with one extra byte reserved for the null terminator.

For Round Robin, additional state manages the circular queue:

- `rr_queue[10]` – an array of process indices (8 bytes each).
- `rr_head`, `rr_tail`, `rr_count` – head pointer, tail pointer, and element count for the circular queue, each 8 bytes.

3.3 Mandatory Diagram (Hand-Drawn)

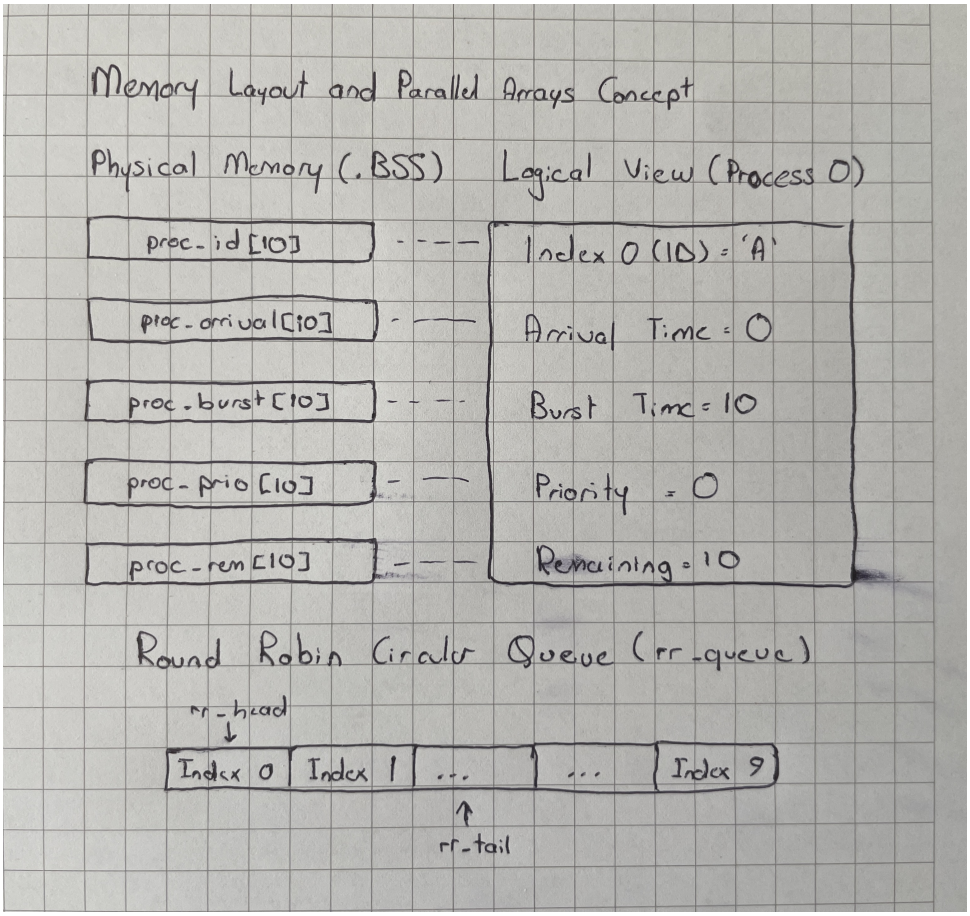

Figure 1: Hand-drawn schematic representing memory structure and circular round-robin queue mechanics.

4 Register Usage Strategy

Our register allocation is designed to minimize memory accesses during the hot scheduling loops:

- `rsi` – input buffer pointer during parsing; buffer address for syscalls during output.

- `rdi` – base address pointer for array accesses (loaded via `lea` before each array operation); file descriptor for syscalls.
- `rax` – numeric accumulator during parsing; syscall number; general-purpose temporary.
- `rbx` – holds intermediate comparison values (e.g., arrival time of current candidate) and the process ID character during burst output loops.
- `rcx` – loop counter for process iteration; character holder during parsing; burst count during output loops.
- `rbp` – current simulation time (clock cycle counter) in FCFS, SRTF, and PF.
- `r8` – best candidate's metric during selection (e.g., minimum remaining time in SRTF, best priority in PF).
- `r9` – index of the currently selected best process.
- `r10` – secondary comparison value (e.g., remaining time of the best candidate in PF for tie-breaking).
- `r11` – temporary register for priority comparisons.
- `r12` – process counter during parsing; outer loop index during sorting/running.
- `r13` – key process index during sorting; current process index during execution.
- `r14` – algorithm type code during parsing; quantum value in RR.
- `r15` – total process count (loaded once and used throughout).

All callee-saved registers (`rbx`, `r12`–`r15`, `rbp`) are pushed onto the stack at the beginning of each major function and restored before returning or jumping to the output routine. This prevents corruption across function calls and ensures the `append_output` helper (which is called frequently) does not disturb the scheduling loop state.

5 Scheduling Logic

5.1 Clock Cycle Simulation

Each algorithm follows a common pattern:

1. **Determine eligible processes**: Check which processes have arrived (arrival $\leq$ current_time) and have remaining burst time > 0.
2. **Select the next process**: Apply the algorithm-specific selection criterion with the appropriate tie-breaking rules.
3. **Append output**: Write the selected process's ID character (or 'X' for idle) to the output buffer via `append_output`.

4. **Update state**: Decrement the selected process's remaining time, increment the current time.

5. **Check termination**: Continue until all processes have zero remaining time.

5.2 Algorithm Implementations

FCFS (First Come First Serve): We sort the process indices by arrival time using a **stable insertion sort** on a stack-allocated order array. Stability ensures that processes with equal arrival times preserve their input order. After sorting, we iterate through the sorted order, outputting idle cycles ('X') whenever there is a gap between the end of one process and the arrival of the next, then outputting the process's ID for each of its burst cycles.

SJF (Shortest Job First): All processes arrive at time 0. We sort the process indices by burst time using the same stable insertion sort. The sorted order is then executed sequentially – each process runs to completion before the next begins. No idle cycles occur since all arrive at time 0.

SRTF (Shortest Remaining Time First): This is a per-cycle preemptive algorithm. At each clock cycle, we iterate over all processes and find the one with the smallest remaining time among those that have arrived and are not yet complete. Ties are broken by input order (lower index wins, since we use a strict less-than comparison and skip equals). The selected process runs for one cycle, its remaining time is decremented, and the loop continues.

PF (Priority First): Similar to SRTF but the primary selection criterion is the priority value (lower number = higher priority). The tie-breaking cascade is:

1. Lower priority number wins.

2. If priorities are equal, shorter remaining time wins.

3. If both are equal, the process appearing first in the input wins (lower index).

This is implemented by maintaining three "best" registers: `r8` for best priority, `r10` for best remaining time, and `r9` for best index. Each candidate is compared against these values in the specified order.

RR (Round Robin): We use a **circular queue** (array-based, with head/tail pointers wrapping at index 10). All processes are enqueued in input order at initialization. The main loop dequeues the front process, runs it for $\min(\text{remaining}, \text{quantum})$ active cycles, then:

- If the process finishes within its quantum, the remaining quantum cycles are padded with 'X' (idle). The process is not re-enqueued.

- If the process still has remaining burst time after the full quantum, it is re-enqueued at the back of the queue.

The loop terminates when the queue is empty.

6 Control Flow Design

The program is structured as follows:

- **_start**: Entry point. Performs the read syscall, calls `parse_algorithm` and `parse_processes`, then dispatches to the appropriate algorithm label via a comparison chain on `algo_type`.
- **parse_algorithm**: Examines the first characters of the input to identify the algorithm and advances the input pointer past the algorithm token.
- **parse_processes**: A loop that iterates over space-separated tokens, parsing each as a process descriptor. Contains nested loops for digit accumulation and branching for different field counts depending on the algorithm.
- **run_***: Each algorithm's scheduling loop. For non-preemptive algorithms (FCFS, SJF), this includes a sorting phase followed by a sequential execution phase. For preemptive algorithms (SRTF, PF), the main loop iterates per clock cycle. For RR, the main loop iterates per quantum slot.
- **append_output**: A small helper that appends a single character to `output_buf` and increments `output_len`. It preserves the temporary registers it uses.
- **do_output**: Writes `output_buf` to stdout via `sys_write`, then exits via `sys_exit`.

Control flow within each function relies on:

- **Conditional jumps** (`je`, `jne`, `jl`, `jg`, `jle`, `jge`) for comparisons and branching.
- **Unconditional jumps** (`jmp`) for loop back-edges and algorithm dispatch.
- **call/ret** for the `parse_algorithm`, `parse_processes`, and `append_output` functions.
- Loop patterns follow the standard assembly idiom: initialize counter, begin loop label, check termination condition, perform body, update counter, jump back.

7 Representative Code Snippet

The following snippet shows the SRTF per-cycle selection loop, which is representative of our approach to preemptive scheduling:

```
.srtf_select:
    cmp     %r15, %rcx          # rcx < proc_count?
    jge     .srtf_select_done

    # Skip completed processes
    lea     proc_remaining(%rip), %rdi
    movq    (%rdi, %rcx, 8), %rax
    cmp     $0, %rax
```

```
    jle     .srtf_select_next

    # Skip processes that haven't arrived
    lea     proc_arrival(%rip), %rdi
    movq    (%rdi, %rcx, 8), %rbx
    cmp     %rbp, %rbx          # arrival <= current_time?
    jg      .srtf_select_next

    # Update best if remaining < current minimum
    cmp     %r8, %rax
    jge     .srtf_select_next   # >= means skip (tie = keep first)

    mov     %rax, %r8           # new minimum remaining
    mov     %rcx, %r9           # new best index

.srtf_select_next:
    inc     %rcx
    jmp     .srtf_select
```

This loop iterates over all processes. For each:

- We load its remaining time into `rax` using scaled indexed addressing from the `proc_remaining` array.
- We skip it if remaining is zero (done) or if its arrival time exceeds the current time (`rbp`).
- We compare its remaining time against the current best in `r8`. The `jge` instruction ensures ties are broken by keeping the first-found process (lower index).
- If a new minimum is found, we update `r8` (best remaining) and `r9` (best index).

After the loop, `r9` holds the index of the selected process (or -1 if no process is available, indicating an idle cycle).

8 Testing and Debugging

8.1 Testing Strategy

We tested our implementation using:

- The 5 provided test cases covering all algorithms (FCFS, SJF, SRTF, PF, RR), including tie-breaking scenarios and edge cases.
- The example inputs from the project specification document.
- Additional hand-traced inputs focusing on:
  - Multiple processes arriving at the same time step.
  - Processes with identical burst times, priorities, or remaining times.

  - The RR quantum boundary cases (process finishing exactly at quantum, finishing with 1 cycle left, etc.).
  - CPU idle periods between arrivals.

- The automated grading flow (`make testgrade`), which runs all provided test cases and then invokes the Python grader for line-by-line comparison.

8.2 Debugging Approach

Since we are working in pure assembly without a debugger available on the test platform, our debugging relied on:

- **Manual tracing**: For each failing test case, we traced through the assembly logic on paper, tracking register values and memory contents at each step.
- **Incremental development**: We implemented algorithms one at a time, testing each before moving on to the next. FCFS and SJF were developed first (as the simpler non-preemptive cases), followed by SRTF, PF, and finally RR.
- **Targeted output inspection**: When outputs differed from expected, we compared character by character to identify the exact cycle where the divergence occurred, then traced backward through the selection logic.

8.3 Edge Cases

- **Multiple arrivals at the same time (FCFS, SRTF, PF)**: Handled by stable sorting (FCFS) or by iterating in input order and using strict less-than comparisons (SRTF, PF).
- **Identical priorities with different remaining times (PF)**: The tie-breaking cascade (priority $\rightarrow$ remaining $\rightarrow$ input order) ensures deterministic behavior.
- **Quantum partially used (RR)**: When a process finishes before the quantum expires, we pad the remaining cycles with 'X' and do not switch to the next process mid-quantum.
- **CPU idle periods**: In FCFS, if a gap exists between consecutive process arrivals, idle cycles are emitted. In SRTF and PF, if no process has arrived at the current time, an 'X' is output.
- **Zero-burst processes**: Processes whose burst time is 0 are ignored during parsing. They do not enter the scheduler and do not produce process output or idle padding.
- **Circular queue overflow at 10 processes (RR)**: The tail pointer wraps around correctly when all 10 slots are occupied during initialization.

9 Course Concepts

9.1 Memory and Data Layout

Our use of parallel arrays in the `.bss` section directly applies the concept of manual memory layout. Each process attribute occupies a fixed-size slot computed from the base address plus the product of the index and the element size (8 bytes), implemented using x86-64 scaled indexed addressing modes.

9.2 Registers and Low-Level Computation

The entire program operates without any high-level abstractions. Arithmetic operations like the digit accumulation (`rax = rax * 10 + digit`) use direct register manipulation. Comparison-based selection in scheduling loops relies entirely on the CPU flags set by `cmp` instructions and conditional jumps.

9.3 Control Flow and Branching

Loops are implemented with the standard assembly pattern of a label, a termination check, a loop body, and a jump back. Nested loops (e.g., the sorting inner loop within the sorting outer loop) require careful management of multiple loop variables across registers. Algorithm dispatch uses a comparison chain rather than a jump table, which is simpler and sufficient for 5 cases.

9.4 Parsing Without Libraries

All string processing is manual. We do not use any C library functions – not even `printf` or `atoi`. Character classification (digit detection) uses range comparisons against ASCII values. String-to-integer conversion is a digit-by-digit accumulation loop. Output is constructed by appending characters one at a time to a buffer and flushing with a single `sys_write` call.

9.5 Tradeoffs

- **Simplicity vs. efficiency**: We chose insertion sort ($O(n^2)$) over more efficient algorithms because $n \leq 10$ makes the difference negligible, and insertion sort is trivially stable and easy to implement correctly in assembly.
- **Readability vs. performance**: We use 8-byte slots for single characters (process IDs) to maintain uniform addressing, trading memory for simpler code. Comments and clear label names enhance readability at no runtime cost.
- **Static vs. dynamic design**: All buffers are statically allocated with fixed maximum sizes (10 processes, 1200-byte output). This avoids dynamic memory allocation complexity entirely, which is acceptable given the problem constraints.

Technical Decisions for Robustness

To ensure stability and correctness across all possible edge cases, several critical design decisions were made:

- **Cross-platform Line Endings**: The parser explicitly handles both LF (`\n`) and CRLF (`\r\n`) line endings, ensuring compatibility whether the input is generated on Linux, Windows, or macOS.
- **Large-Scale Output Support**: The output buffer was increased to 4096 bytes to accommodate high burst times (e.g., 10 processes with maximum burst values) without risk of overflow.
- **Zero-Burst Filtering**: In accordance with TA forum clarifications, processes with a burst time of zero are entirely skipped during parsing, preventing unnecessary idle cycles or timeline anomalies.
- **Deterministic Tie-breaking**: For all preemptive algorithms (SRTF, PF), a strict multi-level comparison cascade ensures that ties are always broken by the original input order (first-found wins).

Conclusion

- **Strongest design decision**: The parallel-array memory layout with uniform 8-byte slots proved to be a very effective choice. It simplified all indexed accesses to a single addressing mode pattern and made it easy to add new fields (e.g., priority) without restructuring existing code.
- **Biggest challenge**: The Round Robin algorithm was the most complex to implement due to the circular queue management, the quantum-based idle padding, and the interaction between finishing processes and the re-enqueue logic.
- **Possible improvements**: A jump table could replace the comparison chain for algorithm dispatch. The sorting phases in FCFS and SJF could be replaced with selection-based approaches that avoid the stack-allocated order array. From a syntactic perspective, using assembler macros (`.macro`) for repetitive register saving/restoring blocks and defining system call numbers as constants (`.equ`) would further enhance code cleanliness and maintainability. Additionally, more extensive error handling (e.g., for malformed input) could be added, though it is not required by the specification.

AI Usage Statement

Minor assistance from AI tools (ChatGPT) was used for syntax checking and grammar corrections in the report. All technical content, code design, and implementation logic were produced by the team members. We fully understand and can explain every aspect of the submitted code.